Stability Analysis of Cyber–Physical Systems Under Wi-Fi Network Congestion

BTech Project Presentation

Shruti Ghoniya

Department of Electrical Engineering

November 25, 2025

Motivation

- Modern robots rely on wireless links for sensing + control.
- Wi-Fi networks introduce latency, jitter, and packet delays.
- Control systems assume timely sensor feedback.
- Core question: **When does Wi-Fi latency begin to break a real-time control loop?**

Initial Work: Leader–Follower Simulation

Goal: Study how Wi-Fi latency affects coordinated robot motion.

- UDP commands: {`dx, dy, duration, seq, timestamp`}
- Two Python processes acting as Leader and Follower
- Performed clock sync (NTP-style)
- Measured:
  - Network latency
  - Command arrival jitter
  - Motion tracking gap

Key Finding

Position error increased with distance from router (Loopback: 0.099 m $\rightarrow$ Far: 0.387 m).

Why This Approach Was Insufficient

- Produced useful latency data but did not stress a real control loop.
- No physics, no real timing sensitivity (no PID derivative term).
- Needed a system that **fails catastrophically** under network delay.

Decision

Shift to a **Hardware-in-the-Loop (HIL)** inverted pendulum experiment.

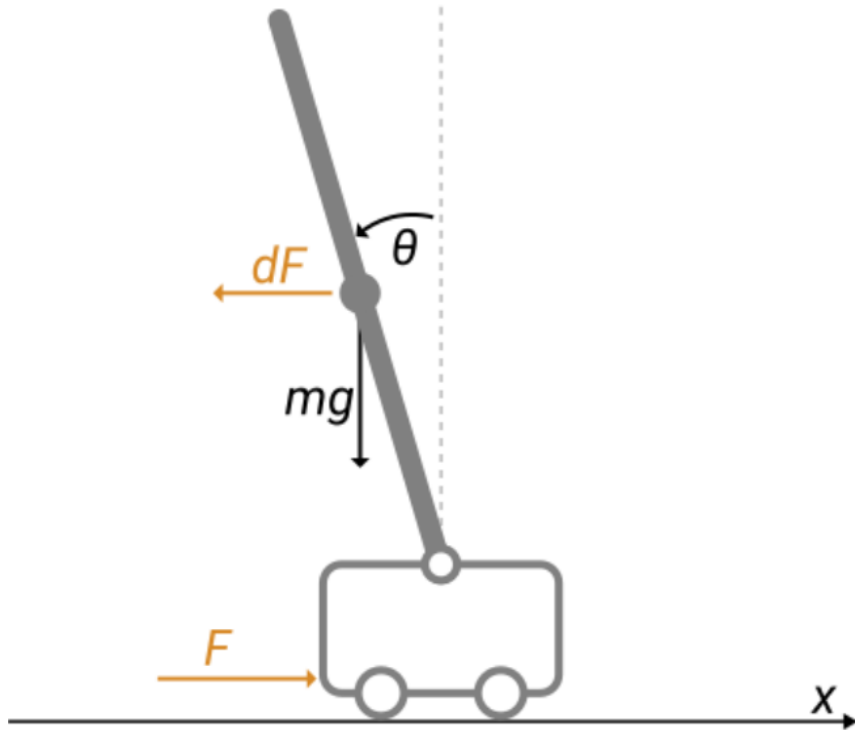
HIL System Overview

Distributed Across Two Wi-Fi Nodes

- Node A – Robot Simulation (100 Hz)
- Node B – PID Controller
- Interference Generator – `iperf3` TCP load

Traffic

- UDP Telemetry (100 Hz, 64 bytes)
- UDP Commands
- TCP Bulk Interference (0–50 Mbps)



Inverted Pendulum Dynamics

$$\ddot{\theta} = \frac{mgl}{I} \sin\theta + \frac{1}{I}\tau \tag{1}$$

Discrete update (10 ms step):

$$\theta_{t+1} = \theta_t + \dot{\theta}_t \Delta t + \frac{1}{2}\ddot{\theta}_t \Delta t^2$$

Control Law (PID):

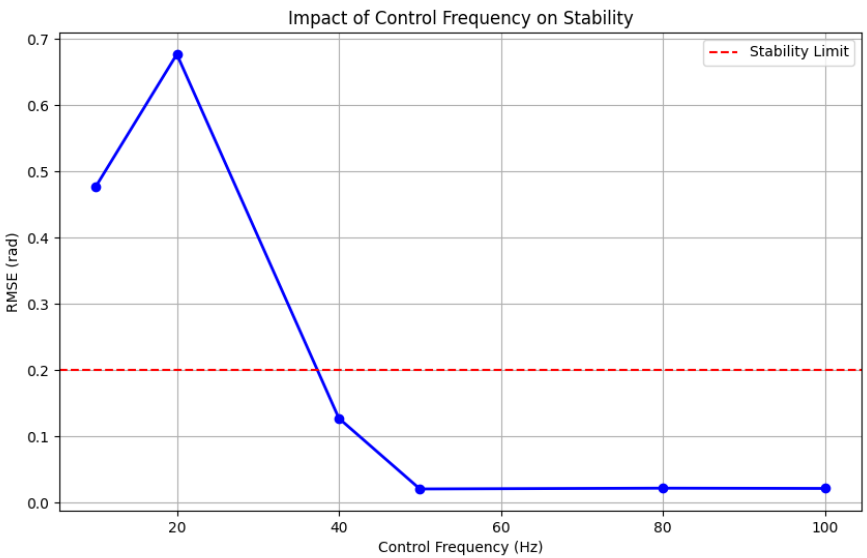
$$u_t = K_p e_t + K_i \sum e_t \Delta t + K_d \frac{e_t - e_{t-1}}{\Delta t_{\text{arrival}}}$$

Why Derivative is Dangerous

If packets arrive in a burst:

$$\Delta t_{\text{arrival}} \to 0 \Rightarrow D \to \infty$$

Characterization of Control Frequency Sensitivity



Stability vs. Packet Rate (10-100 Hz)

Objective: Observe system limits under reduced packet rates (simulating throughput throttling).

Key Observations:

- **Stability Cliff:** The system fails immediately below 40 Hz.
- **Cause:** Discrete time steps at low frequencies introduce fatal Phase Lag.

Baseline Selection

100 Hz sits safely within the stable region, providing a robust buffer against minor fluctuations.

Traffic Flow Configuration

To isolate failure modes, we varied the source of the background traffic (TCP) while maintaining the bidirectional control loop (UDP).

Scenario A: Uplink Stress

Aggressor: The Robot

- **Robot Sends:**
  - UDP Telemetry (Signal)
  - 20 Mbps TCP (Noise)
- **Controller Sends:**
  - UDP Command (Signal)

Scenario B: Downlink Stress

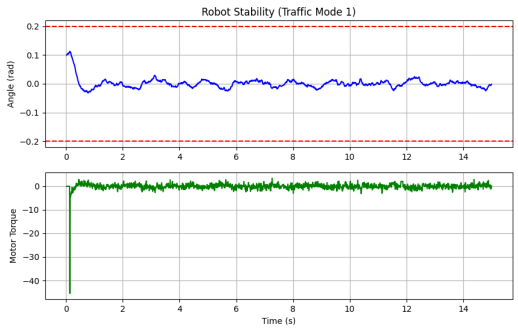
Aggressor: The Controller

- **Controller Sends:**
  - UDP Command (Signal)
  - 50 Mbps TCP (Noise)
- **Robot Sends:**
  - UDP Telemetry (Signal)

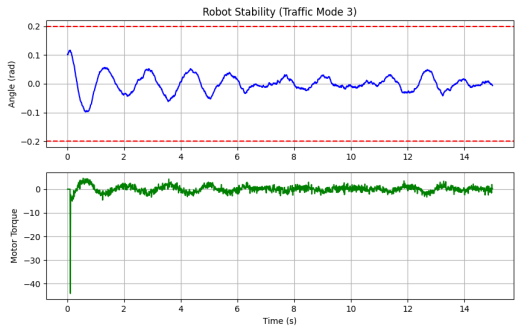
Key Distinction: In both cases, the UDP loop is closed (bidirectional). However, the **TCP flood is unidirectional**, congesting only the outgoing path of the Aggressor.

Scenario Comparison

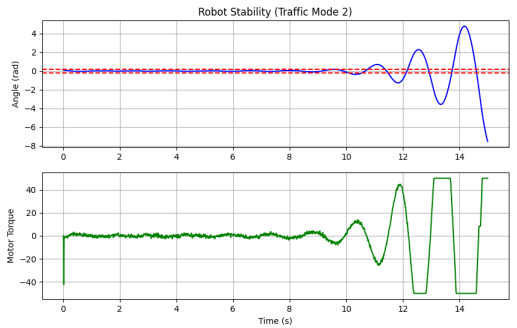
- **Baseline (0 Mbps)** – Stable, smooth.
- **Downlink Stress (50 Mbps)** Jitter → vibration but no fall.
- **Uplink Stress (20 Mbps)** Queue fills → stale telemetry → robot collapses.



Baseline



Downlink Stress



Uplink Stress

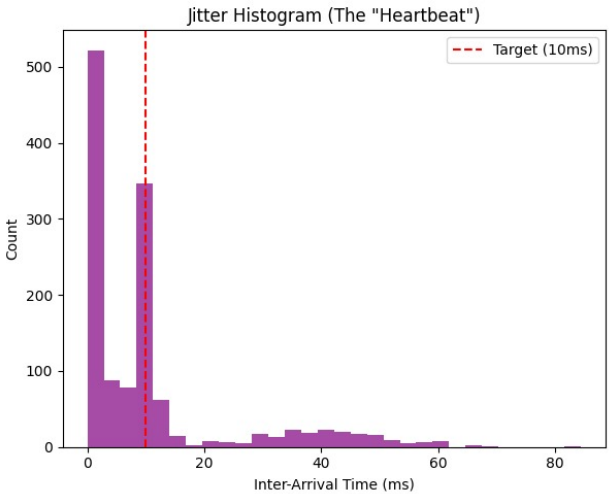
Stability Threshold Sweep

TCP Load	RMSE	Eff. Loss	Status
0 Mbps	0.002	0%	Stable
15 Mbps	0.045	0%	Stable
18 Mbps	0.120	0.6%	Marginal
30 Mbps	> 1.0	2.45%	**Failed**

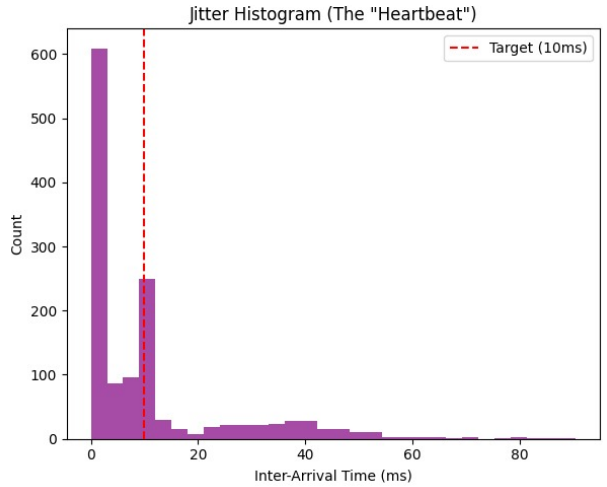
Critical Finding

18 Mbps is the stability boundary. Beyond this, the robot fell in all the instances.

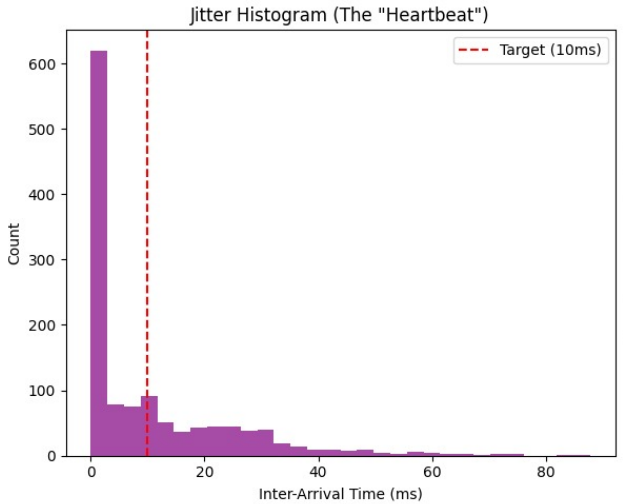
Count vs Interarrival time Transition Across Load Levels



15 Mbps (Stable)



18 Mbps (Marginal)



30 Mbps (Failure)

- 15 Mbps $\rightarrow$ clean single peak at 10ms.
- 18 Mbps $\rightarrow$ long-tail jitter appears.
- 30 Mbps $\rightarrow$ bimodal pattern $\rightarrow$ **clumping effect**.

Evolution of the 10ms Heartbeat

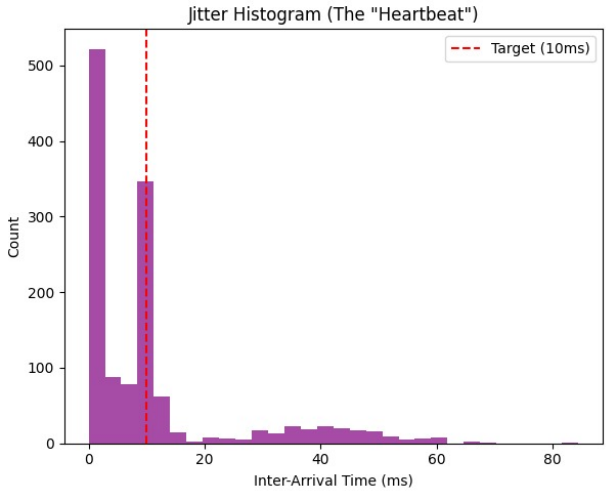
Baseline: Robot transmits strictly at 100 Hz ($\Delta t = 10$ms).

- **15 Mbps (Safe Zone)** Dominant 10ms peak $\rightarrow$ UDP finds immediate TX slots.
- **18 Mbps (Transition)** 10ms bar shrinks; a long tail ($>$20 ms) emerges $\rightarrow$ queue buildup begins.
- **30 Mbps (Failure)** 10ms peak nearly disappears; jitter becomes bimodal (0–2 ms spike + 80–100 ms tail) $\rightarrow$ **Packet Clumping**.

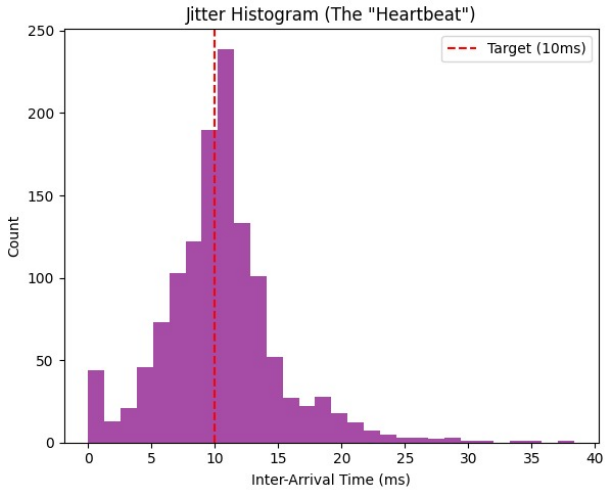
Interpretation

Increasing congestion shifts arrival times from deterministic to chaotic, eventually collapsing the regular 10ms heartbeat.

TCP vs. UDP Interference



TCP: Structured "Pulse"



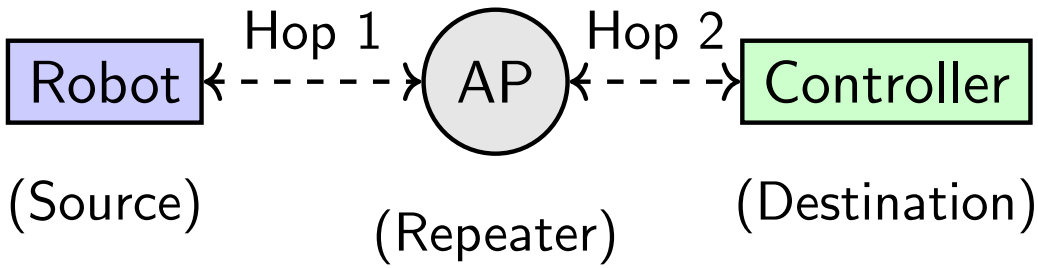
UDP: Random "Noise"

- **TCP:** Sends a window of packets, then pauses to wait for ACKs
  - *Result:* **Bimodal Histogram.** During ACK pauses, the queue empties, allowing the **native 10ms signal** to pass. During Bursts, packets are trapped.
- **UDP:** Open-loop Streaming.
  - *Mechanism:* Constant pressure. No ACKs, no pauses.
  - *Result:* **"Bell Curve" Jitter.** The queue remains semi-full, adding stochastic delay to *every* packet.

The Setup

Topology: Wireless Robot $\leftrightarrow$ Access Point $\leftrightarrow$ Wireless Controller.
In Infrastructure Mode, frames must traverse the air interface twice.

1. **Half-Duplex:** Only one node (Robot, Controller, or AP) can transmit at a time.
2. **Shared Bandwidth:** Throughput is effectively halved.



Scenario 1: Uplink Stress (Robot Uploads 20 Mbps TCP)

Latency on Feedback Path → System Failure

Configuration:

- Robot generates **20 Mbps** TCP.
- Bottleneck: Robot's NIC Queue.

Packet Fate Analysis:

1. **Telemetry (Rob→Ctrl):** High Latency. Trapped in Robot's NIC Queue behind TCP.
2. **Command (Ctrl→Rob):** High Jitter. Controller must fight for airtime against Robot/AP.

Symmetry of Failure

The **Telemetry** here suffers the exact same **Queuing Delay** as the Commands in Scenario 2.

Outcome: Fall

Stale Sensor Data → Controller corrects for past error → Divergence.

Scenario 2: Downlink Stress (Ctrl Uploads 50 Mbps TCP)

Jitter on Feedback Path $\rightarrow$ System Vibration

Configuration:

- Controller generates **50 Mbps** TCP.
- Bottleneck: Controller's NIC Queue.

Packet Fate Analysis:

1. **Command (Ctrl$\rightarrow$Rob):** High Latency. Trapped in Controller's NIC Queue.
2. **Telemetry (Rob$\rightarrow$Ctrl):** High Jitter. Robot waits (Backoff) for AP/Ctrl to stop transmitting.

Symmetry of Failure

The **Commands** here suffer the exact same **Queuing Delay** as the Telemetry in Scenario 1.

Outcome: Shake

Noisy Sensor Data $\rightarrow$ Derivative Spike ($D \rightarrow \infty$) $\rightarrow$ High-Freq Vibration.

Literature Validation

Theoretical frameworks verify that delay location matters:

$$\tau_{total} = \tau_{sc} \text{ (Uplink/Sensor)} + \tau_{ca} \text{ (Downlink/Actuator)}$$

Sensor Delay (τ_{sc})

Destabilizes Estimator

- Delayed system data
- **Mechanism:** Derivative term (D) calculated on old data creates "False Velocity".
- **Result:** Immediate Instability.

Actuator Delay (τ_{ca})

Adds Phase Lag

- Estimation remains accurate.
- **Mechanism:** Correct command applied late.
- **Result:** Oscillation (can be stabilized via tuning).

Reference: Hespanha et al. (IEEE 2007)

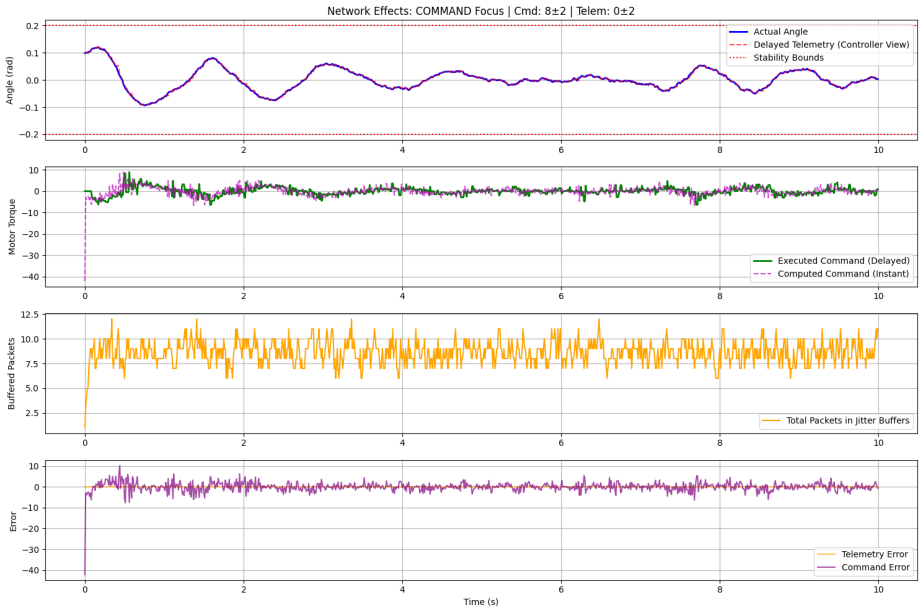
"Networked Control Systems: A Brief Overview" - Feedback path delays (τ_{sc}) are more destructive than forward path delays due to estimation error.

Root Cause Validation: Synthetic Delay Injection

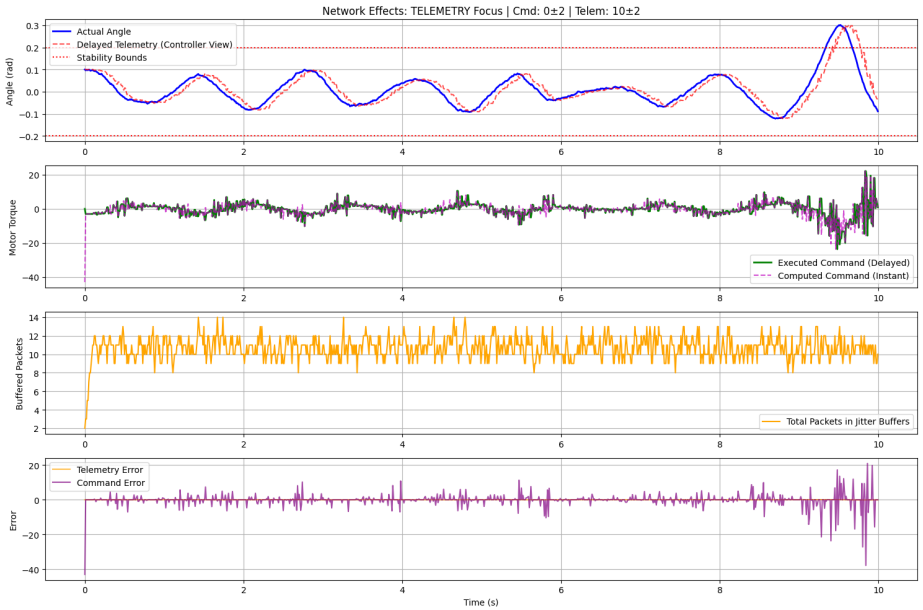
To verify that **Telemetry Delay** is the dominant failure mode, we ran a standalone simulation using **fixed baseline PID gains**.

Note: This experiment validates structural sensitivity using standard gains; exhaustive tuning for delay compensation wasn't performed.

A. Command Delay



B. Telemetry Delay



Observation: Stable

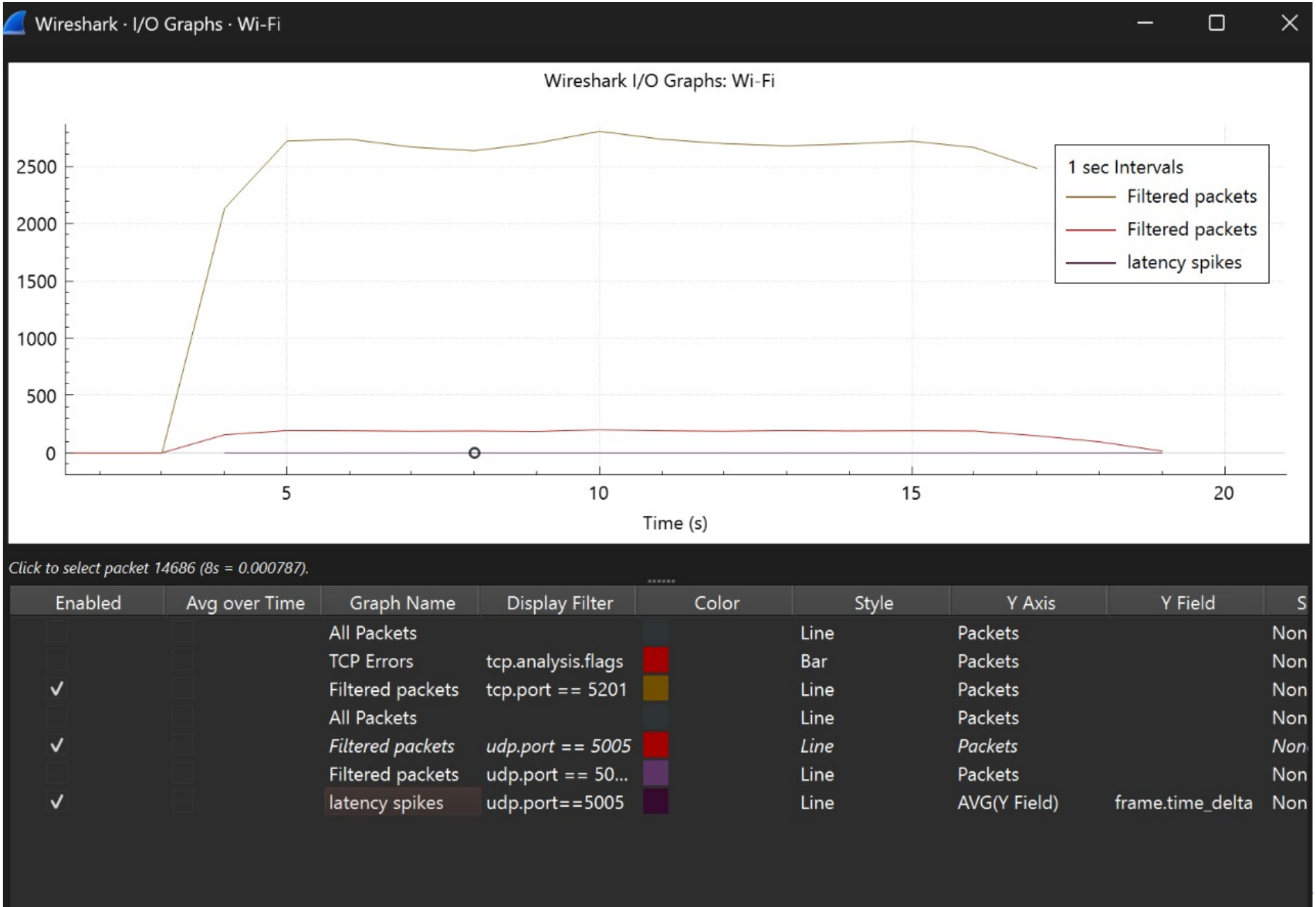
With standard gains, Command Delay introduces Phase Lag (Green torque is delayed), yet the system maintains bounded stability.

Observation: Unstable

With the same gains, Telemetry Delay causes the Controller View (Red) to desynchronize from Reality (Blue). **Result: Immediate Divergence.**

Conclusion: Even without specific retuning, the system is demonstrably more sensitive

Wireshark Validation



Key Takeaways

- Wi-Fi congestion breaks closed-loop stability before packet loss occurs.
- Identified operational limit: **18 Mbps**.

Core Answer

Instead of just packet losses, latency can also be a bottleneck in Wi-Fi-based robotic control.

Experimental Insight: Efficient stability is not a binary choice but a multi-variable "Sweet Spot" that requires exhaustive experimentation:

- **Control Logic:** Tuning PID gains to tolerate phase lag.
- **Network Hygiene:** Capping background traffic to prevent queue saturation.
- **Temporal Resolution:** Fixing an optimum frequency .